TP – Budget étudiant

Niveau moyen – directives de base, v-model, computed

Pr. Ahmed Bentajer

Cahier des charges

Une petite application de suivi de budget personnel, à destination d'un étudiant : on enregistre ses dépenses au fil de l'eau, on les filtre par catégorie, et on surveille un budget mensuel fixé à l'avance.

1 Le formulaire d'ajout

Complétez le `<form>` : liez chaque champ au bon attribut de `nouvelleDepense` avec `v-model` (`.titre`, `.montant`, `.categorie`), et ajoutez `@submit.prevent="ajouterDepense"` sur la balise `<form>` elle-même.

2 Filtrer par catégorie

Dans la `computed depensesFiltrees`, renvoyez `this.depenses` telles quelles si `filtreCategorie` vaut `'toutes'`, sinon seulement celles dont la catégorie correspond. Complétez aussi `selectionnerCategorie(cat)` et dans le template, donnez à chaque bouton de filtre son `@click` et sa classe `active` (inspirez-vous du bouton « Toutes », déjà écrit).

3 Totaux

Complétez `totalFiltre` (somme des montants de `depensesFiltrees`), `totalGeneral` (somme de **toutes** les dépenses, indépendamment du filtre), et `moyenneDepense` (`totalFiltre` divisé par le nombre de dépenses filtrées).

Faites attention !!

Pourquoi deux totaux différents ? Parce que l'alerte de budget (section suivante) doit réagir au total **réel** de toutes les dépenses, pas seulement à celles actuellement affichées par le filtre – sans quoi filtrer sur « Loisirs » ferait *disparaître* une alerte de dépassement pourtant bien réelle. Pensez aussi à gérer la division par zéro dans `moyenneDepense` : une liste filtrée vide ne doit pas afficher `NaN`.

4 Alerte de budget : v-if / v-else-if / v-else

Remplacez les trois lignes du résumé par une vraie chaîne conditionnelle, basée sur `totalGeneral` comparé à `budgetMensuel` : sous 80 % du budget (ok), entre 80 % et 100 % (attention), au-delà (dépassé).

Faites attention !!

`v-else-if` et `v-else` doivent être portés par des éléments **directement adjacents** à l'élément `v-if` dans le template – aucun autre élément, même un simple texte ou un commentaire HTML visible, ne doit s'intercaler entre les trois balises. C'est une source d'erreur fréquente dès qu'on réorganise un peu le template.

5 Détail par catégorie : v-show

Le bouton « Afficher le détail par catégorie » doit inverser `afficherStats` ; le panneau qui suit doit lui être lié par `v-show`. Complétez aussi la méthode `totalCategorie(cat)` (même principe que `totalFiltre`, mais restreint à une seule catégorie), et appelez-la dans le `v-for` du panneau.

Pour info

`v-show` laisse l'élément dans le DOM et bascule juste sa visibilité CSS (`display: none`), alors que `v-if` le retire et le recrée entièrement. Pour un panneau qu'on ouvre/ferme souvent, comme ici, `v-show` évite de refaire ce travail à chaque clic – c'est pour ça qu'on l'utilise plutôt que `v-if` à cet endroit précis.

6 La liste et sa suppression

Bouclez sur `depensesFiltrees` (avec `:key="d.id"`), affichez la liste seulement si elle n'est pas vide (`v-if`/`v-else` avec le message déjà en place), complétez `ajouterDepense()` et `supprimerDepense(depense)`.

Aller plus loin

- Ajoutez un champ `date` à chaque dépense, et un tri « plus récent d'abord » dans l'affichage (sans utiliser `sort` : une boucle qui construit le tableau dans le bon ordre suffit).
- Permettez de modifier le `budgetMensuel` directement depuis l'interface, avec `v-model` sur un champ dédié.